

MI SSL API

Version 1.2

© 2021 SigmaStar Technology Corp. All rights reserved.

SigmaStar Technology makes no representations or warranties including, for example but not limited to, warranties of merchantability, fitness for a particular purpose, non-infringement of any intellectual property right or the accuracy or completeness of this document, and reserves the right to make changes without further notice to any products herein to improve reliability, function or design. No responsibility is assumed by SigmaStar Technology arising out of the application or use of any product or circuit described herein; neither does it convey any license under its patent rights, nor the rights of others.

SigmaStar is a trademark of SigmaStar Technology Corp. Other trademarks or names herein are only for identification purposes only and owned by their respective owners.

REVISION HISTORY

Revision No.	Description	Date
1.0	● Initial release	09/12/2020
1.1	● Modified API	10/15/2020
1.2	● 4MIC Sound Source Localization and Modified API	11/03/2021

TABLE OF CONTENTS

REVISION HISTORY	i
TABLE OF CONTENTS	ii
1. 概述	1
1.1. 算法说明	1
1.2. 关键词说明	1
1.3. 注意	1
1.3.1. 使用	1
1.3.2. 区分	2
2. 数组系统及坐标系	3
2.1. 多麦克风数组	3
2.1.1. 均匀线性数组及坐标系	3
2.1.2. 均匀圆形数组及坐标系	3
3. API 参考	5
3.1. 功能模块 API	5
3.2. IaaSsl_GetBufferSize	5
3.3. IaaSsl_Init	6
3.4. IaaSsl_Config	7
3.5. IaaSsl_Get_Config	8
3.6. IaaSsl_Set_Shape	8
3.7. IaaSsl_Cal_Params	9
3.8. IaaSsl_Run	10
3.9. IaaSsl_Get_Direction	19
3.10. IaaSsl_Reset_Mapping	20
3.11. IaaSsl_Reset	21
3.12. IaaSsl_Free	22
4. SSL 数据类型	24
4.1. SSL 模块相关数据类型定义	24
4.2. AudioSslInit	24
4.3. AudioSslConfig	25
4.4. SSL_HANDLE	26

5. 错误码	27
---------------	-----------

1. 概述

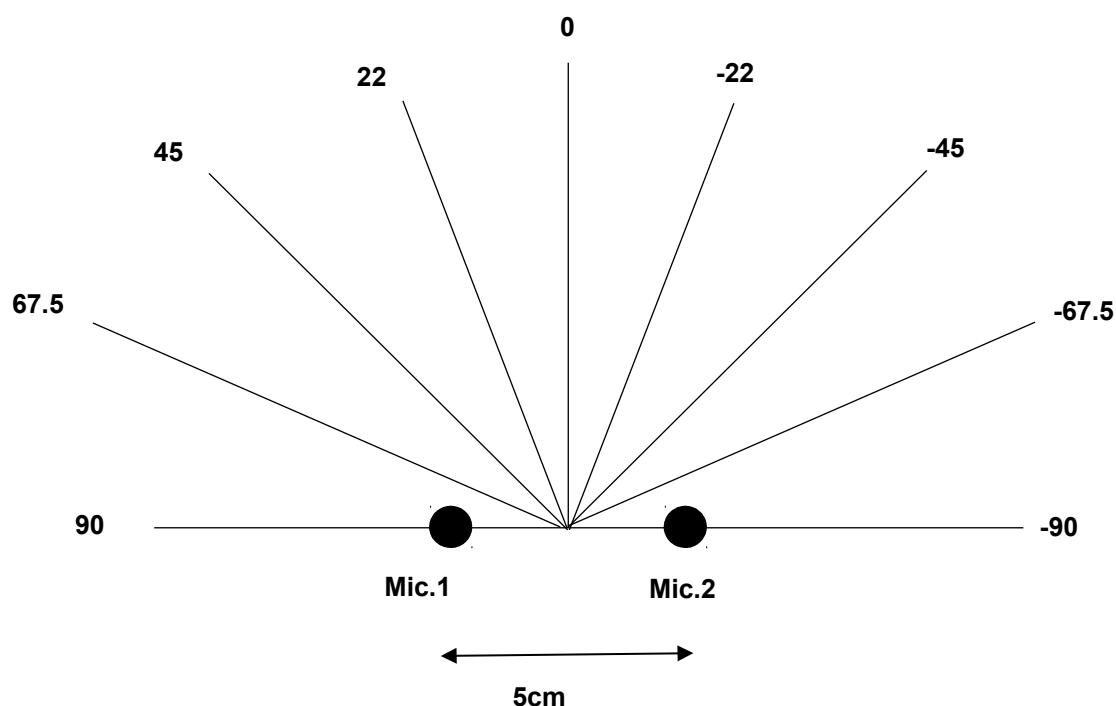
1.1. 算法说明

SSL (Sound Source Localization) 声源定位，用于定位声音来源的方向。

1.2. 关键词说明

● Direction

当两个麦克风距离(distance)为 5cm 时，下图描述音源方向(direction)来源的定义：



当麦克风个数多于两颗以上，将会有两种可能的数组形状，分别是均匀线性数组以及均匀圆形数组。

1.3. 注意

1.3.1. 使用

为方便调试和确认算法效果，需要用户应用自行实现替换算法参数和抓取音频数据的逻辑。

1.3.2. 区分

不同通道数会对应到不同参考库。用户需要自行确认是否使用正确的通道数及对应之参考库。

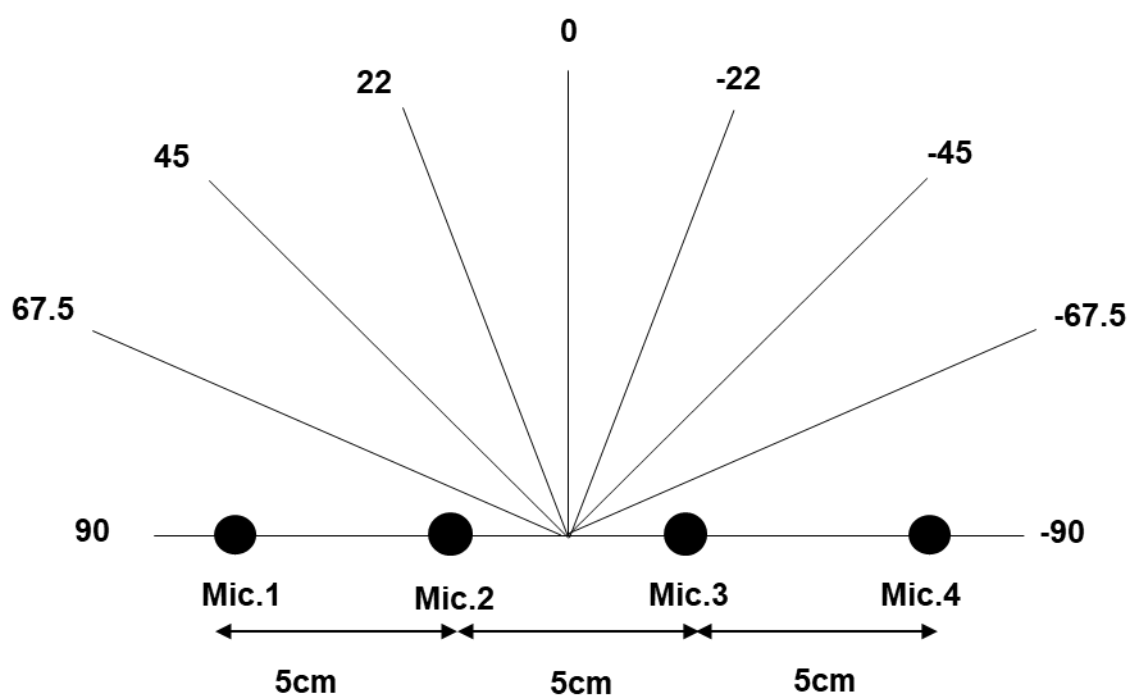
2. 数组系统及坐标系

2.1. 多麦克风数组

常见的多麦克风数组主要有两类，分别是均匀线性数组以及均匀圆形数组。以下皆为二维示意图(无高度差之影响)。

2.1.1. 均匀线性数组及坐标系

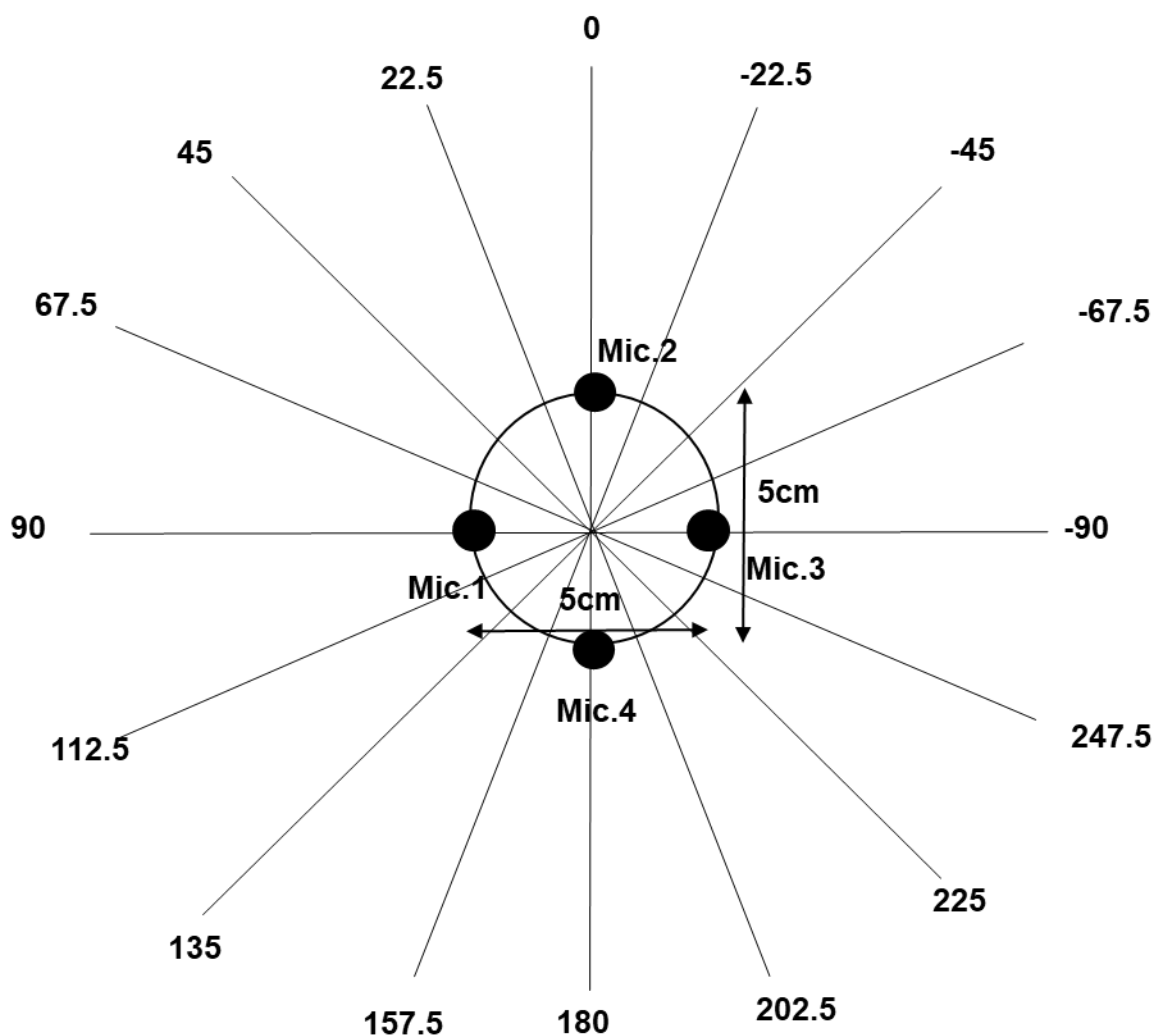
均匀线性数组为一直线排列，且相邻麦克风之间距 **distance** 皆相同，并且由于其具有对称性，定位角度仅考虑上半平面(-90 度至 90 度)。图二为均匀线性数组于空间坐标系之示意图。声音方向的角度 (**microphone_doa**) 定义为与数组中心及 **x** 轴之夹角。逆时针方向为正。**distance** 单位为公分，建议至少大于 5 cm 或 6 cm。



图二 均匀线性数组及其坐标系

2.1.2. 均匀圆形数组及坐标系

均匀圆形数组为一圆形排列，且相邻麦克风与数组中心之夹角皆相同，麦克风间距即为圆形之直径，由于不具对称性，定位角度可以为整个平面(-90 度至 270 度)，图三为均匀圆形数组于空间坐标系之示意图。声音方向的角度 (**microphone_doa**) 定义为与数组中心及 **x** 轴之夹角。逆时针方向为正。**distance** 为圆形的直径。单位为公分。建议至少大于 6 cm。



图三 均匀圆形数组及其坐标系

3. API 参考

3.1. 功能模块 API

API 名	功能
IaaSsl_GetBufferSize	获取 Ssl 算法运行需要的内存大小
IaaSsl_Init	初始化 Ssl 算法
IaaSsl_Config	配置 Ssl 算法
IaaSsl_Get_Config	获取 Ssl 算法当前的配置参数信息
IaaSsl_Set_Shape	Ssl 指派麦克风数组形状
IaaSsl_Cal_Params	SSL 根据数组形状计算参数

API 名	功能
IaaSsl_Run	Ssl 算法处理
IaaSsl_Get_Direction	获取 Ssl 算法处理的结果
IaaSsl_Reset_Mapping	当 IaaSsl_Get_Direction 完成后重新初始化定位缓冲区
IaaSsl_Reset	重新初始化 Ssl 算法
IaaSsl_Free	释放 Ssl 算法资源

3.2. IaaSsl_GetBufferSize

➤ 功能

获取 Ssl 算法运行所需要的内存大小。

➤ 语法

```
unsigned int IaaSsl_GetBufferSize(void);
```

➤ 形参

参数名称	描述	输入/输出
------	----	-------

➤ 返回值

返回值为 Ssl 算法运行所需要的内存大小，此内存大小与使用的麦克风数有关。

➤ 依赖

- 头文件: AudioSslProcess.h
- 库文件: libSSL_2MIC_LINUX.so/ libSSL_2MIC_LINUX.a/libSSL_4MIC_LINUX.so/libSSL_4MIC_LINUX.a

※ 注意

该接口仅返回需要的内存大小，申请和释放内存的动作需应用来处理。

➤ 举例

请参考 [IaaSsl_Run](#) 举例部分。

3.3. IaaSsl_Init

➤ 功能

初始化 Ssl 算法。

➤ 语法

```
SSL_HANDLE IaaSsl_Init(char* working_buffer, AudioSslInit* ssl_init);
```

➤ 形参

参数名称	描述	输入/输出
working_buffer	Ssl 算法所使用的内存地址，用户申请内存大小后所得地址	输入
ssl_init	Ssl 算法的初始化结构体指针	输入

➤ 返回值

返回值	结果
非 NULL	成功
NULL	失败

➤ 依赖

- 头文件：AudioSslProcess.h
- 库文件：libSSL_2MIC_LINUX.so/ libSSL_2MIC_LINUX.a/libSSL_4MIC_LINUX.so/libSSL_4MIC_LINUX.a

※ 注意

无。

➤ 举例

请参考 [IaaSsl_Run](#) 举例部分。

3.4. IaaSsl_Config

➤ 功能

配置 Ssl 算法。

➤ 语法

```
ALGO_SSL_RET IaaSsl_Config(SSL_HANDLE handle, AudioSslConfig* ssl_config);
```

➤ 形参

参数名称	描述	输入/输出
handle	Ssl 算法 handle	输入
ssl_config	Ssl 算法的配置结构体指针	输入

➤ 返回值

返回值	结果
0	成功
非 0	失败，参照 错误码

➤ 依赖

- 头文件: AudioSslProcess.h
- 库文件: libSSL_2MIC_LINUX.so/ libSSL_2MIC_LINUX.a/libSSL_4MIC_LINUX.so/libSSL_4MIC_LINUX.a

※ 注意

无。

➤ 举例

请参考 [IaaSsl_Run](#) 举例部分。

3.5. IaaSsl_Get_Config

➤ 功能

获取 Ssl 算法当前的配置参数。

➤ 语法

[ALGO_SSL_RET](#) IaaSsl_Get_Config([SSL_HANDLE](#) handle,[AudioSslConfig](#) *ssl_config);

➤ 形参

参数名称	描述	输入/输出
handle	Ssl 算法 handle	输入
ssl_config	Ssl 算法的配置结构体指针	输出

➤ 返回值

返回值	结果
0	成功
非 0	失败, 参照 错误码

➤ 依赖

- 头文件: AudioSslProcess.h
- 库文件: libSSL_2MIC_LINUX.so/ libSSL_2MIC_LINUX.a/libSSL_4MIC_LINUX.so/libSSL_4MIC_LINUX.a

※ 注意

无。

➤ 举例

请参考 [IaaSsl_Run](#) 举例部分。

3.6. IaaSsl_Set_Shape

➤ 功能

SSL 指派数组形状。

➤ 语法

[ALGO_SSL_RET](#) IaaSsl_Set_Shape([SSL_HANDLE](#) handle,[int](#) shape);

➤ 形参

参数名称	描述	输入/输出
handle	Ssl 算法 handle	输入
shape	指派数组形状。0:均匀线性数组, 1:均匀圆形数组	输入

➤ 返回值

返回值	结果
0	成功
非 0	失败, 参照 错误码

➤ 依赖

- 头文件: AudioSslProcess.h
- 库文件: libSSL_2MIC_LINUX.so/ libSSL_2MIC_LINUX.a/libSSL_4MIC_LINUX.so/libSSL_4MIC_LINUX.a

※ 注意

- 仅提供均匀线性数组及均匀圆型数组(示意图请看图二及图三)，客户有特殊数组需求，需告知修改
- 麦克风个数为 2 个时，只存在线性数组，其间距即为所设定麦克风间距
- 数组位置设定会强烈影响 SSL 效果，因此设定之数组位置必须与真实数组位置相同

➤ 举例

请参考 [IaaSsl_Run](#) 举例部分。

3.7. IaaSsl_Cal_Params

➤ 功能

SSL 根据数组形状计算其定位用之参数。

➤ 语法

```
ALGO_SSL_RET IaaSsl_Cal_Params(SSL_HANDLE handle);
```

➤ 形参

参数名称	描述	输入/输出
handle	Ssl 算法 handle	输入

➤ 返回值

返回值	结果
0	成功
非 0	失败，参照 错误码

➤ 依赖

- 头文件: AudioSslProcess.h
- 库文件: libSSL_2MIC_LINUX.so/ libSSL_2MIC_LINUX.a/libSSL_4MIC_LINUX.so/libSSL_4MIC_LINUX.a

※ 注意

无。

➤ 举例

请参考 [IaaSsl_Run](#) 举例部分。

3.8. IaaSsl_Run

➤ 功能

Ssl 算法处理

➤ 语法

```
ALGO_SSL_RET IaaSsl_Run(SSL_HANDLE handle, short* microphone_input, int *delay_sample);
```

➤ 形参

参数名称	描述	输入/输出
handle	Ssl 算法 handle	输入
microphone_input	待进行声源定位的数据指针	输入
delay_sample	不同对麦克风之间的延迟样本数。此值建议当 bf_mode 开启时再拿作使用。	输出

➤ 返回值

返回值	结果
0	成功
非 0	失败，参照 错误码

➤ 依赖

- 头文件: AudioSslProcess.h
- 库文件: libSSL_2MIC_LINUX.so/ libSSL_2MIC_LINUX.a/libSSL_4MIC_LINUX.so/libSSL_4MIC_LINUX.a

※ 注意

- 对于两颗麦克风数组(双声道音频可以负荷)，microphone_input 指向的数据应以采样点为最小单位，按照 L,R,L,R 的格式摆放，数据长度必须和 IaaSsl_Init 中设定的 point_number(即多少个采样点进行一次 beamforming 处理)相对应。→ 范例一
- 对于多颗麦克风数组(麦克风数多于两颗，双声道音频无法负荷)之情况，各个麦克风音频须为单声道，microphone_input 指向的数据仍应以采样点为最小单位，并按照麦克风于数组的相对位置，由左至右的格式摆放，例如 4 颗线性麦克风数组，摆放的方式为 [左一→左二→右二→右一]，数据长度必须和 IaaBf_Init 中设定的 point_number(即多少个采样点进行一次 SSL 处理)相对应。→ 范例二
- 以图二及图三之麦克风数组为例，microphone_input 摆放的格式为[MIC1→MIC2→MIC3→MIC4].

➤ 举例

- 范例一：两颗麦克风数组(麦克风输入为双声道音频)

```
1. #include <stdio.h>
2. #include <unistd.h>
3. #include <fcntl.h>
4. #include <string.h>
5. #include <sys/time.h>
6. #include <sys/ioctl.h>
7. #include <stdlib.h>
8. #include "AudioSslProcess.h"
9.
10. #define MIC_NUM (2)
11. #define USE_MALLOC (1)
12. typedef unsigned char      uint8;
13. typedef unsigned short     uint16;
14. typedef unsigned long      uint32;
15.
16.
17. unsigned int _OsCounterGetMs(void)
18. {
19.     struct timeval t1;
20.     gettimeofday(&t1, NULL);
21.     unsigned int T = ( (1000000 * t1.tv_sec)+ t1.tv_usec ) / 1000;
22.     return T;
23. }
24.
25. int main(int argc, char *argv[])
26. {
27.     /*****Input file init*****/
28.     short input[256];
29.     char inFile[512];
30.     char outFile[512];
31.     FILE * fin;
32.     FILE * fout;
33.     ALGO_SSL_RET ret;
34.     int counter2 = 0;
35.     unsigned int T0,T1,T2,T3;
36.     float avg = 0.0;
37.     float avg2 = 0.0;
38.     /*****common setting SSL*****/
```

```
39. int point_number = 128;
40. float microphone_distance = 12.0;
41. int temperature = 20;
42. int sample_rate = 16000;
43. int delay_sample[1] = {0};
44. int shape = 0;
45. int direction = 0;
46. int frame_number = 32;
47. /*****SSL data init*****/
48. int counter = 0;
49. #if USE_MALLOC
50. char *WorkingBuffer2;
51. WorkingBuffer2 = (char*)malloc(IaaSsl_GetBufferSize());
52. #endif
53. AudioSslInit ssl_init;
54. AudioSslConfig ssl_config;
55. SSL_HANDLE handle;
56.
57. ssl_init.mic_distance = microphone_distance; //cm
58. ssl_init.point_number = point_number;
59. ssl_init.sample_rate = sample_rate;
60. ssl_init.bf_mode = 0;
61. ssl_init.channel = MIC_NUM;
62. ssl_config.temperature = temperature; //c
63. ssl_config.noise_gate_dbfs = -80;
64. ssl_config.direction_frame_num = frame_number;
65. /*****init algorithm*****/
66. handle = IaaSsl_Init((char*)WorkingBuffer2, &ssl_init);
67. if (handle==NULL)
68. {
69.     printf("SSL init error\n\r");
70.     return -1;
71. }
72. else
73. {
74.     printf("SSL init succeed\n\r");
75. }
76.
77. ret = IaaSsl_Config(handle, &(ssl_config));
78. if (ret)
79. {
80.     printf("Error occured in SSL Config\n\r");
81.     return -1;
```

```
82. }
83. ret = IaaSsl_Set_Shape(handle,shape);
84. if (ret)
85. {
86.     printf("Error occured in Array shape\n\r");
87.     return -1;
88. }
89.
90. ret = IaaSsl_Cal_Params(handle);
91. if (ret)
92. {
93.     printf("Error occured in Array matrix calculation\n\r");
94.     return -1;
95. }
96.
97. sprintf(infileName,"%s","../sample/data/Chn_Left_right_12_0.wav");
98. sprintf(outfileName,"%s","./SSL_result.txt");
99.
100. fin = fopen(infileName, "rb");
101. if(!fin)
102. {    printf("the input file 0 could not be open\n\r");
103.     return -1;
104. }
105.
106. fout = fopen(outfileName, "w");
107. if(!fout)
108. {
109.     printf("the output file could not be open\n\r");
110.     return -1;
111. }
112.
113. fread(input, sizeof(char), 44, fin); // read header 44 bytes
114. fprintf(fout,"%s\t%s\t%s\n\r","time","direction","case");
115. while(fread(input, sizeof(short), ssl_init.point_number*2, fin))
116. {
117.     counter++;
118.     T0 = (long)_OsCounterGetMs();
119.     ret = IaaSsl_Run(handle,input,delay_sample);
120.     if(ret != 0)
121.     {
122.         printf("The Run fail\n");
123.         return -1;
124.     }
```

```

125.     // low resolution
126.     // if (ssl_init.bf_mode == 1)
127.     // {
128.     //     printf("delay_sample: %d\n",delay_sample[0]);
129.     // }
130.     T1 = (long)_OsCounterGetMs();
131.     avg += (T1-T0);
132.     if(counter == ssl_config.direction_frame_num && ssl_init.bf_mode == 0)
133.     {
134.         counter2++;
135.         counter= 0;
136.         T2 = (long)_OsCounterGetMs();
137.         ret = IaaSsl_Get_Direction(handle, &direction);
138.         T3 = (long)_OsCounterGetMs();
139.         avg2 += (T3-T2);
140.         if(ret != 0 && ret!=ALGO_SSL_RET_RESULT_UNRELIABLE && ret!
=ALGO_SSL_RET_BELOW_NOISE_GATE&&ret!
=ALGO_SSL_RET_DELAY_SAMPLE_TOO_LARGE)
141.         {
142.             printf("The Get_Direction fail\n");
143.             return -1;
144.         }
145.         // write txt file
146.         fprintf(fout, "%f\t%d", (float)
(counter2*ssl_config.direction_frame_num*0.008), direction);
147.         if (ret==0)
148.         {
149.             fprintf(fout, "\t%s\n\r", "current time is reliable!");
150.         }
151.         else if (ret==ALGO_SSL_RET_BELOW_NOISE_GATE)
152.         {
153.             fprintf(fout, "\t%s\n\r", "current time volume is too small!");
154.         }
155.         else if (ret==ALGO_SSL_RET_DELAY_SAMPLE_TOO_LARGE)
156.         {
157.             fprintf(fout, "\t%s\n\r", "current time delay_sample is out of range!");
158.         }
159.         else
160.         {
161.             fprintf(fout, "\t%s\n\r", "current time is not reliable!");
162.         }
163.         // reset voting
164.         ret = IaaSsl_Reset_Mapping(handle);

```

```

165.         if(ret != 0)
166.         {
167.             printf("The ResetVoting fail\n");
168.             return -1;
169.         }
170.     }
171. }
172. avg = avg / (float)(ssl_config.direction_frame_num*counter2);
173. avg2 = avg2 / (float)(counter2);
174. printf("AVG for IaaSSL_RUN is %.3f ms\n",avg);
175. printf("AVG for IaaSSL_GetDirection is %.3f ms\n",avg2);
176. IaaSsl_Free(handle);
177. fclose(fin);
178. fclose(fout);
179. free(WorkingBuffer2);
180. printf("Done\n");
181. return 0;
182. }

```

- 范例二：多颗麦克风数组(麦克风输入为4个单声道音讯)

```

1.  #include <stdio.h>
2.  #include <unistd.h>
3.  #include <fcntl.h>
4.  #include <string.h>
5.  #include <sys/time.h>
6.  #include <sys/ioctl.h>
7.  #include <stdlib.h>
8.  #include "AudioSslProcess.h"
9.
10. #define MIC_NUM (4)
11. #define USE_MALLOC (1)
12. typedef unsigned char      uint8;
13. typedef unsigned short     uint16;
14. typedef unsigned long      uint32;
15.
16. unsigned int _OsCounterGetMs(void)
17. {
18.     struct timeval t1;
19.     gettimeofday(&t1,NULL);
20.     unsigned int T = ( (1000000 * t1.tv_sec)+ t1.tv_usec ) / 1000;
21.     return T;

```

```
22. }
23.
24. int main(int argc, char *argv[])
25. {
26.     /*****Input file init*****/
27.     short input[512];
28.     short input_tmp1[128],input_tmp2[128],input_tmp3[128],input_tmp4[128];
29.     char inFileName[MIC_NUM][512];
30.     char outFileName[512];
31.     FILE * fin0,* fin1,* fin2,* fin3;
32.     FILE * fout;
33.     int k;
34.     ALGO_SSL_RET ret;
35.     int counter2 = 0;
36.     unsigned int T0,T1,T2,T3;
37.     float avg = 0.0;
38.     float avg2 = 0.0;
39.     /*****common setting SSL *****/
40.     int point_number = 128;
41.     float microphone_distance = 4.0;
42.     int temperature = 20;
43.     int sample_rate = 16000;
44.     int delay_sample[MIC_NUM-1] = {0,0,0}; //channel-1
45.     int shape = 0;
46.     int direction = 0;
47.     int frame_number = 32;
48.     /*****SSL data init*****/
49.     int counter = 0;
50.     #if USE_MALLOC
51.     char *WorkingBuffer_SSL;
52.     WorkingBuffer_SSL = (char*)malloc(IaaSsl_GetBufferSize());
53.     #endif
54.     AudioSslInit ssl_init;
55.     AudioSslConfig ssl_config;
56.     SSL_HANDLE ssl_handle;
57.
58.     ssl_init.mic_distance = microphone_distance;
59.     ssl_init.point_number = point_number;
60.     ssl_init.sample_rate = sample_rate;
61.     ssl_init.bf_mode = 0;
62.     ssl_init.channel = MIC_NUM;
63.     ssl_config.temperature = temperature;
64.     ssl_config.noise_gate_dbfs = -80;
```

```
65.  ssl_config.direction_frame_num = frame_number;
66.
67.  /*****init algorithm *****/
68.  ssl_handle = IaaSsl_Init((char*)WorkingBuffer_SSL, &ssl_init);
69.  if (ssl_handle == NULL)
70.  {
71.      printf("Init fail\n\r");
72.      return -1;
73.  }
74.  else
75.  {
76.      printf("SSL init succeed\n\r");
77.  }
78.
79.  ret = IaaSsl_Config(ssl_handle, &(ssl_config));
80.  if (ret)
81.  {
82.      printf("Error occured in SSL Config\n\r");
83.      return -1;
84.  }
85.
86.  ret = IaaSsl_Set_Shape(ssl_handle, shape);
87.  if (ret)
88.  {
89.      printf("Error occured in Array shape\n\r");
90.      return -1;
91.  }
92.
93.  ret = IaaSsl_Cal_Params(ssl_handle);
94.  if (ret)
95.  {
96.      printf("Error occured in Array matrix calculation\n\r");
97.      return -1;
98.  }
99.
100.  /*****open input file and input file*****/
101.
102.  sprintf(infileName[0], "%s", "../sample/data/Chn-01.wav");
103.  sprintf(infileName[1], "%s", "../sample/data/Chn-02.wav");
104.  sprintf(infileName[2], "%s", "../sample/data/Chn-03.wav");
105.  sprintf(infileName[3], "%s", "../sample/data/Chn-04.wav");
106.  sprintf(outfileName, "%s", "../SSL_result.txt");
107.  fin0 = fopen(infileName[0], "rb");
```

```
108.     if(!fin0)
109.     {
110.         printf("the input file0 could not be open\n\r");
111.         return -1;
112.     }
113.     fin1 = fopen(infileName[1], "rb");
114.     if(!fin1)
115.     {
116.         printf("the input file 1 could not be open\n\r");
117.         return -1;
118.     }
119.     fin2 = fopen(infileName[2], "rb");
120.     if(!fin2)
121.     {
122.         printf("the input file 2 could not be open\n\r");
123.         return -1;
124.     }
125.     fin3 = fopen(infileName[3], "rb");
126.     if(!fin3)
127.     {
128.         printf("the input file 3 could not be open\n\r");
129.         return -1;
130.     }
131.     fout = fopen(outfileName, "w");
132.     if(!fout)
133.     {
134.         printf("the output file could not be open\n\r");
135.         return -1;
136.     }
137.
138.     fread(input, sizeof(char), 44, fin0); // read header 44 bytes
139.     fread(input, sizeof(char), 44, fin1); // read header 44 bytes
140.     fread(input, sizeof(char), 44, fin2); // read header 44 bytes
141.     fread(input, sizeof(char), 44, fin3); // read header 44 bytes
142.
143.     short * input_ptr;
144.     fprintf(fout, "%s\t%s\t%s\n\r", "time", "direction", "case");
145.     while(fread(input_tmp1, sizeof(short), point_number, fin0))
146.     {
147.         fread(input_tmp2, sizeof(short), point_number, fin1);
148.         fread(input_tmp3, sizeof(short), point_number, fin2);
149.         fread(input_tmp4, sizeof(short), point_number, fin3);
150.         input_ptr = input;
```

```

151.     for(k=0;k<point_number;k++)
152.     {
153.         *input_ptr = input_tmp1[k];
154.         input_ptr++;
155.         *input_ptr = input_tmp2[k];
156.         input_ptr++;
157.         *input_ptr = input_tmp3[k];
158.         input_ptr++;
159.         *input_ptr = input_tmp4[k];
160.         input_ptr++;
161.     }
162.     counter++;
163.     T0 = (long)_OsCounterGetMs();
164.     ret = IaaSsl_Run(ssl_handle,input,delay_sample);
165.     if(ret != 0)
166.     {
167.         printf("The Run fail\n");
168.         return -1;
169.     }
170.     // low resolution
171.     // if (ssl_init.bf_mode == 1)
172.     // {
173.     //     printf("delay_sample: %d,%d,%d\n",delay_sample[0],delay_sample[1],delay_sample[2]);
174.     // }
175.     T1 = (long)_OsCounterGetMs();
176.     avg += (T1-T0);
177.
178.     if(counter == ssl_config.direction_frame_num && ssl_init.bf_mode == 0)
179.     {
180.         counter2++;
181.         counter= 0;
182.         T2 = (long)_OsCounterGetMs();
183.         ret = IaaSsl_Get_Direction(ssl_handle, &direction);
184.         T3 = (long)_OsCounterGetMs();
185.         avg2 += (T3-T2);
186.         if(ret != 0 && ret!=ALGO_SSL_RET_RESULT_UNRELIABLE && ret!
            =ALGO_SSL_RET_BELOW_NOISE_GATE&&ret!
            =ALGO_SSL_RET_DELAY_SAMPLE_TOO_LARGE)
187.         {
188.             printf("The Get_Direction fail\n");
189.             return -1;
190.         }

```

```
191.         // write txt file
192.         fprintf(fout, "%f\t%d", (float)
(counter2*ssl_config.direction_frame_num*0.008), direction);
193.         if (ret==0)
194.         {
195.             fprintf(fout, "\t%s\n\r", "current time is reliable!");
196.         }
197.         else if (ret==ALGO_SSL_RET_BELOW_NOISE_GATE)
198.         {
199.             fprintf(fout, "\t%s\n\r", "current time volume is too small!");
200.         }
201.         else if (ret==ALGO_SSL_RET_DELAY_SAMPLE_TOO_LARGE)
202.         {
203.             fprintf(fout, "\t%s\n\r", "current time delay_sample is out of range!");
204.         }
205.         else
206.         {
207.             fprintf(fout, "\t%s\n\r", "current time is not reliable!");
208.         }
209.         // reset voting
210.         ret = IaaSsl_Reset_Mapping(ssl_handle);
211.         if (ret != 0)
212.         {
213.             printf("The ResetVoting fail\n");
214.             return -1;
215.         }
216.     }
217. }
218. avg = avg / (float)(ssl_config.direction_frame_num*counter2);
219. avg2 = avg2 / (float)(counter2);
220. printf("AVG for IaaSSL_RUN is %.3f ms\n", avg);
221. printf("AVG for IaaSSL_GetDirection is %.3f ms\n", avg2);
222. IaaSsl_Free(ssl_handle);
223. fclose(fin0);
224. fclose(fin1);
225. fclose(fin2);
226. fclose(fin3);
227. fclose(fout);
228. free(WorkingBuffer_SSL);
229. printf("Done\n");
230.
231. return 0;
232. }
```

3.9. IaaSsl_Get_Direction

➤ 功能

获取 Ssl 算法的处理结果

➤ 语法

ALGO_SSL_RET IaaSsl_Get_Direction(**SSL_HANDLE** handle, **int*** direction);

➤ 形参

参数名称	描述	输入/输出
handle	Ssl 算法 handle	输入
direction	，若是均匀线性数组，数值介于-90~90 之间，若是均匀圆形数组，则数值介于-90~270 之间，当回传数值为-10000，代表可能发生下面三种状况其中之一。第一种状况为音量低于 noise_gate_dbfs ；第二种情况为数据量不够，无法计算出可信的方向；第三种状况为估测状况超出范围	输出

➤ 返回值

返回值	结果
0	成功
0x10000107	成功。警告：估测状况超出范围
0x10000113	成功。警告：音量低于噪音阈值
0100000114	成功。警告：数据量不足以计算出可信的方向
其他	失败，参照 错误码

➤ 依赖

- 头文件：AudioSslProcess.h
- 库文件：libSSL_2MIC_LINUX.so/ libSSL_2MIC_LINUX.a/libSSL_4MIC_LINUX.so/libSSL_4MIC_LINUX.a

※ 注意

当呼叫 [IaaSsl_Get_Direction](#) 后务必呼叫 IaaSsl_Reset_Mapping。

➤ 举例

请参考 [IaaSsl_Run](#) 举例部分。

3.10. IaaSsl_Reset_Mapping

➤ 功能

当 IaaSsl_Get_Direction 完成后重新初始化定位缓冲区。

➤ 语法

```
ALGO_SSL_RET IaaSsl_Reset_Mapping(SSL_HANDLE handle);
```

➤ 形参

参数名称	描述	输入/输出
handle	Ssl 算法 handle	输入

➤ 返回值

返回值	结果
0	成功
非 0	失败，参照 错误码

➤ 依赖

- 头文件: AudioSslProcess.h
- 库文件: libSSL_2MIC_LINUX.so/ libSSL_2MIC_LINUX.a/libSSL_4MIC_LINUX.so/libSSL_4MIC_LINUX.a

※ 注意

当呼叫 [IaaSsl_Get_Direction](#) 后务必呼叫 IaaSsl_Reset_Mapping

➤ 举例

请参考 [IaaSsl_Run](#) 举例部分。

3.11. IaaSsl_Reset

➤ 功能

重新初始化 Ssl 算法。

➤ 语法

```
SSL_HANDLE IaaSsl_Reset(SSL_HANDLE working_buffer, AudioSslInit* ssl_init);
```

➤ 形参

参数名称	描述	输入/输出
working_buffer	Ssl 算法运行的内存地址	输入
ssl_init	Ssl 算法的初始化参数结构体指针	输入

➤ 返回值

返回值	结果
0	成功
非 0	失败，参照 错误码

➤ 依赖

- 头文件：AudioSslProcess.h
- 库文件：libSSL_2MIC_LINUX.so/ libSSL_2MIC_LINUX.a/libSSL_4MIC_LINUX.so/libSSL_4MIC_LINUX.a

※ 注意

无。

➤ 举例

请参考 [IaaSsl_Run](#) 举例部分。

3.12. IaaSsl_Free

➤ 功能

释放 Ssl 算法资源

➤ 语法

```
ALGO_SSL_RET IaaSsl_Free(SSL_HANDLE handle);
```

➤ 形参

参数名称	描述	输入/输出
handle	Src 算法 handle	输入

➤ 返回值

返回值	结果
0	成功
非 0	失败, 参照 错误码

➤ 依赖

- 头文件: AudioSslProcess.h
- 库文件: libSSL_2MIC_LINUX.so/ libSSL_2MIC_LINUX.a/libSSL_4MIC_LINUX.so/libSSL_4MIC_LINUX.a

※ 注意

无。

➤ 举例

请参考 [IaaSsl_Run](#) 举例部分。

4. SSL 数据类型

4.1. SSL 模块相关数据类型定义

数据类型	定义
AudioSslInit	Ssl 算法初始化参数结构体类型
AudioSslConfig	Ssl 算法配置参数结构体类型
SSL_HANDLE	Ssl 算法句柄类型

4.2. AudioSslInit

➤ 说明

定义 Ssl 算法的初始化参数类型。

➤ 定义

```
typedef struct
{
    unsigned int point_number;
    unsigned int sample_rate;
    float mic_distance;
    unsigned int bf_mode;
```

```
int channel;
}AudioSslInit;
```

➤ 成员

成员名称	描述
point_number	Ssl 算法处理一次的采样点数
sample_rate	取样率，目前支持 8k/16k/32k/48k
mic_distance	两个 mic 的距离，单位为 cm
bf_mode	是否为 beamforming 模式 若想使用定位角度，设定为 0 并由 IaaSsl_Get_Direction 得到定位结果。若想使用估测的延迟的样本数，设定为 1
channel	麦克风个数

※ 注意事项

- 若需 delay_sample 则建议将 bf_mode 开启。
- 若将 bf_mode 设为 1，则无法呼叫 IaaSsl_Get_Direction 以及 IaaSsl_Reset_Mapping。
- 若数组为均匀线性数组，则 mic_distance 需设定为相邻麦克风之间的距离，若为均匀圆形数组，mic_distance 需设定为圆形的直径。

➤ 相关数据类型及接口

IaaSsl_Init、IaaSsl_Reset

4.3. AudioSslConfig

➤ 说明

定义 Ssl 算法的配置参数结构体类型。

➤ 定义

```
typedef struct
{
    unsigned int temperature;
    int noise_gate_dbfs;
    int direction_frame_num;
}AudioSslConfig;
```

➤ 成员

成员名称	描述
temperature	环境温度（摄氏度） 摄氏度 = $(5/9) * (\text{华氏度} - 32)$ 步长为 1
noise_gate_dbfs	噪音增益门限值（dBfs） 注意：低于此值将会把该帧作为噪声部分处理，则不会进入方向的计算。 步长为 1
direction_frame_num	声源定位功能检测的帧数 步长为 32 注意：进行声源定位检测的帧数，数值必须为 32 的倍数，声源定位处理的一帧数据为 128 个采样点。检测一次的时间 = $s32DirectionFrameNum * 128 / \text{采样率}$ 。如：当前采样率为 16K，s32DirectionFrameNum 设置成 32，则检测时间 = $32 * 128 / 16000 = 0.256(s)$

※ 注意事项

- 无。

➤ 相关数据类型及接口

[IaaSsl_Config](#)、[IaaSsl_Get_Config](#)

4.4. SSL_HANDLE

➤ 说明

定义 Ssl 算法的句柄类型。

➤ 定义

```
typedef void* SSL_HANDLE;
```

➤ 成员

成员名称	描述
------	----

※ 注意事项

- 无

➤ 相关数据类型及接口

[IaaSsl_Init](#)、[IaaSsl_Config](#)、[IaaSsl_Get_Config](#)、[IaaSsl_Set_Shape](#)、[IaaSsl_Cal_Params](#)、[IaaSsl_Run](#)、[IaaSsl_Get_Direction](#)、[IaaSsl_Reset_Mapping](#)、[IaaSsl_Reset](#)、[IaaSsl_Free](#)

5. 错误码

SSL API 错误码如下表所示：

表 5-1 SSL API 错误码

错误码	宏定义	描述
0x00000000	ALGO_SSL_RET_SUCCESS	SSL 执行成功
0x10000101	ALGO_SSL_RET_INIT_ERROR	SSL 初始化错误
0x10000102	ALGO_SSL_RET_INVALID_CONFIG	SSL Config 无效
0x10000103	ALGO_SSL_RET_INVALID_HANDLE	SSL Handle 无效
0x10000104	ALGO_SSL_RET_INVALID_SAMPLERATE	SSL 采样率无效
0x10000105	ALGO_SSL_RET_INVALID_POINTNUMBER	SSL 采样点数无效
0x10000106	ALGO_SSL_RET_INVALID_BFMODE	SSL init 的 bf_mode 设置无效
0x10000107	ALGO_SSL_RET_DELAY_SAMPLE_TOO_LARGE	警告:延迟样本过大, 请检查设定的距离和取样率
0x10000108	ALGO_SSL_RET_INVALID_CALLING	SSL 呼叫 API 顺序错误
0x10000109	ALGO_SSL_RET_API_CONFLICT	有其他 API 正在运行
0x10000110	ALGO_SSL_RET_INVALID_CHANNEL	SSL 通道数无效
0x10000111	ALGO_SSL_RET_INVALID_GEOMETRY_TYPE	SSL 数组形状无效
0x10000112	ALGO_SSL_RET_INVALID_ARRAY_TYPE	SSL 双麦克风数组必为线性数组 shape=0
0x10000113	ALGO_SSL_RET_BELOW_NOISE_GATE	警告:音量小于噪音阈值
0x10000114	ALGO_SSL_RET_RESULT_UNRELIABLE	警告:数据量不足以计算出可信的方向